UNIVERSIDAD DE CARTAGENA
Programa de Ingeniería de Software

Sistema de nómina orientado a objetos

ACTIVIDAD UNIDAD TRES
INGENIERÍA DE SOFTWARE

Jesus Davod Campo
Autor

Cartagena de Indias
1 de noviembre de 2026

Repositorio del proyecto
https://github.com/tab-Jesus/IngDeSoftware

Video de exposición

https://drive.google.com/drive/folders/12X3QH36apLfZmf3hXu9CXgjdpZZKqqcB?hl=es

1. Introducción, objetivos y requisitos

El cálculo de una nómina requiere aplicar reglas diferentes según la modalidad de vinculación del empleado. El proyecto resuelve este problema mediante un modelo orientado a objetos que distingue los ingresos, los beneficios, las deducciones y el costo empresarial de cada liquidación. Las reglas funcionales corresponden a la Actividad Unidad Tres de Ingeniería de Software [1].

Objetivo general. Desarrollar un sistema de nómina en Python que calcule los valores de cuatro modalidades de empleado mediante programación orientada a objetos y principios SOLID.

Objetivos específicos. Modelar las modalidades de contratación; separar responsabilidades; validar los datos; calcular importes con precisión decimal; comprobar las reglas mediante pruebas unitarias y exportar resultados a CSV.

Requisito	Comportamiento implementado
Asalariado	Salario fijo mensual y bono del 10% cuando la antigüedad supera cinco años.
Por horas	Hasta 40 horas a tarifa normal; las adicionales se pagan a 1,5 veces la tarifa. Sin bono monetario.
Por comisión	Base más comisión sobre ventas. Bono del 3% sobre todas las ventas cuando superan $20.000.000.
Temporal	Salario fijo y contrato con fechas inicial y final. Sin bonos ni alimentación adicional.
Beneficios	Alimentación de $1.000.000 para permanentes. Ahorro del 2% para empleados por horas con más de un año y aceptación.
Validaciones	Rechazo de horas y ventas negativas, datos no finitos y liquidaciones con neto negativo.
Calidad	Diseño SOLID, funciones con responsabilidades delimitadas, comentarios, refactorización y pruebas unitarias.

Alcance. La aplicación procesa datos de demostración por consola y genera comprobantes desglosados y un reporte CSV. El dominio permite construir otros empleados y calcular nuevos escenarios mediante sus clases y servicios.

2. Reglas de cálculo y criterios del modelo

La liquidación diferencia el salario bruto, el valor neto del empleado y el costo empresarial. Los porcentajes de seguro, pensión y ARL se concentran en una política configurable. Para los ejemplos y las pruebas se adoptaron los siguientes criterios:

Concepto	Criterio aplicado
Seguro y pensión	Se interpreta el porcentaje del enunciado como 4% del bruto para seguro y otro 4% para pensión. Esta distribución es un supuesto del modelo y puede modificarse en DeductionPolicy.
ARL	Aporte empresarial separado del neto, calculado con una tasa ilustrativa configurable del 0,522%.
Alimentación	Beneficio de $1.000.000 para asalariados y empleados por comisión; se registra separado del bruto y del neto.
Ahorro	Retención del 2% del bruto destinada al fondo, únicamente para empleados por horas con más de un año y aceptación expresa.
Período	El umbral de 40 horas se aplica a las horas ingresadas. En contratos temporales, la fecha de liquidación debe estar dentro del intervalo, incluidos sus extremos. Sin prorrateo.
Precisión	Importes con Decimal [2]. Redondeo ROUND_HALF_UP a dos decimales por concepto. El bruto suma el ordinario y el bono previamente redondeados.

Fórmulas generales

Bruto = pago ordinario + bono monetario.
Seguro = bruto × tasa de seguro.
Pensión = bruto × tasa de pensión.
Ahorro = bruto × 2%, cuando aplica.
Neto = bruto - seguro - pensión - ahorro.
Costo empresarial modelado = bruto + alimentación + ARL.

Los límites de antigüedad y ventas se evalúan de forma estricta: exactamente cinco años no generan el bono del asalariado; exactamente un año no habilita el ahorro y ventas de $20.000.000 no activan el bono adicional de comisión.

3. Diseño orientado a objetos

Abstracción. Employee define el contrato común del empleado mediante una clase abstracta de Python [3]. Su operación earnings() representa la obtención de ingresos, y validate_period() verifica la elegibilidad del período.

Herencia y polimorfismo. Salaried, Hourly, Commission y Temporary especializan el contrato. El servicio invoca las mismas operaciones y recibe resultados calculados según la modalidad, sin consultar el tipo concreto del empleado.

Encapsulación. Las entidades validan sus atributos al construirse y usan dataclasses inmutables [4]. De esta manera, los cálculos parten de objetos cuyo nombre, identificación y valores numéricos cumplen las restricciones del modelo.

Elemento	Responsabilidad
Employee	Contrato de ingresos, período, alimentación y tasa de ahorro.
Clases de empleados	Reglas particulares de salario, bono, beneficios y elegibilidad.
DeductionPolicy	Validación de tasas; cálculo de seguro, pensión y ARL empresarial.
PayrollService	Coordinación de ingresos y deducciones; validación del neto y generación del resultado.
Payslip	Comprobante inmutable con ingresos, descuentos, beneficios y costo empresarial.
Consola y CSV	Presentación de ejemplos y exportación del desglose.

Secuencia de liquidación. Primero se valida el período. Después se obtienen y redondean el pago ordinario y el bono para formar el bruto. Se calculan seguro, pensión y ahorro; se verifica que el neto sea no negativo y se incorporan alimentación y ARL al comprobante.

Extensión. Una modalidad adicional puede implementar el contrato Employee y reutilizar PayrollService. La separación entre dominio y salida permite desarrollar otras interfaces sobre las mismas reglas.

4. SOLID, código limpio y refactorización

Principio	Aplicación en el proyecto
S: responsabilidad única	Las entidades calculan ingresos; la política calcula descuentos; el servicio coordina; la consola presenta y exporta.
O: abierto/cerrado	Las nuevas modalidades se incorporan mediante especialización de Employee, sin agregar condicionales por tipo al servicio.
L: sustitución de Liskov	Las modalidades proporcionan las operaciones del contrato común y expresan su elegibilidad mediante validate_period().
I: segregación de interfaces	El contrato del empleado se limita al dominio de nómina y no exige funciones de impresión, persistencia ni interfaz gráfica.
D: inversión de dependencias	PayrollService utiliza el contrato Employee y recibe la política de descuentos por constructor, lo que permite sustituirla por otra compatible.

Organización del código. Se emplearon nombres descriptivos, funciones breves y validaciones próximas a la definición de los datos. Las operaciones amount() y money() centralizan la validación numérica y el redondeo para evitar repetir criterios en cada modalidad.

Comentarios. Las cadenas de documentación explican los contratos de las clases y las decisiones relevantes, como el tratamiento de deducciones y la responsabilidad del servicio. Los comentarios complementan la lectura de los nombres y las operaciones.

Refactorización. La comprobación del período temporal se trasladó al método validate_period() de la entidad. El servicio pasó de identificar una modalidad concreta a solicitar una validación polimórfica. También se separaron las tasas en DeductionPolicy y el resultado en Payslip.

Comprobación del comportamiento. Los resultados esperados de las pruebas se expresan como valores numéricos explícitos. Esto permite contrastar la liquidación con cálculos manuales y comprobar la consistencia del servicio tras la reorganización de responsabilidades.

5. Pruebas unitarias y resultados

La suite de validación contiene 25 pruebas ejecutadas con unittest [5]. Se comprobaron cálculos de las cuatro modalidades, condiciones de frontera, configuración alternativa de tasas, redondeo y rechazo de datos inválidos. El resultado de la ejecución fue de 25 pruebas aprobadas, sin fallos.

Caso de demostración	Bruto	Neto	Costo empresa
Asalariado: $3.000.000; 6 años	$3.300.000	$3.036.000	$4.317.226
Horas: 45 a $20.000; con ahorro	$950.000	$855.000	$954.959
Comisión: base $2.000.000; ventas $25.000.000; tasa 5%	$4.000.000	$3.680.000	$5.020.880
Temporal: salario $1.800.000	$1.800.000	$1.656.000	$1.809.396

Verificación del ejemplo por horas. Las 40 horas normales suman $800.000 y las cinco horas extras suman $150.000. Sobre el bruto de $950.000 se descuentan $38.000 de seguro, $38.000 de pensión y $19.000 de ahorro. El neto es $855.000. La ARL empresarial es $4.959 y el costo modelado de la empresa es $954.959.

Grupo de pruebas	Casos comprobados
Resultados	Valores esperados para asalariado, horas, comisión y temporal; alimentación separada del neto.
Fronteras	Cinco años y antigüedad superior; 40 horas; ventas iguales y superiores a $20.000.000; un año de antigüedad y aceptación del ahorro.
Datos y contrato	Valores negativos y no finitos; nombre vacío; tasas fuera de rango; fechas invertidas, extremas y fuera del contrato.
Consistencia	Neto negativo rechazado; cero permitido; redondeo de 1,005 a 1,01; tasas del 2% y 2% con neto de 96 sobre bruto de 100.

Registro de resultados. evidencia/pruebas.txt conserva la ejecución de la suite y evidencia/nomina_ejemplo.csv contiene el desglose de los cuatro empleados ficticios utilizados en la demostración. Los valores de la tabla están expresados en pesos colombianos.

6. Metodología y control de versiones

El proyecto se desarrolló mediante una metodología incremental. Las reglas del problema se organizaron como requisitos, se implementó el dominio y se incorporaron la salida por consola, la exportación y las pruebas. La documentación consolida el diseño y los resultados de cada incremento.

Etapa	Trabajo realizado y criterio de aceptación
Análisis	Identificación de las cuatro modalidades, beneficios, deducciones y validaciones. Definición explícita de criterios para las tasas y el período.
Diseño e implementación	Construcción de entidades, política, servicio y comprobante. Cálculo de los cuatro ejemplos con separación de responsabilidades.
Verificación	Contraste con valores manuales y ejecución de 25 pruebas. Aceptación de la implementación al obtener los resultados esperados y rechazar las entradas inválidas evaluadas.
Documentación	Descripción de reglas y diseño; organización del informe, la presentación y las instrucciones de ejecución.

Control de versiones. Los cambios del código, las pruebas y la documentación se registraron en Git. El repositorio permite consultar el historial, identificar las modificaciones de cada versión y recuperar estados anteriores. Los materiales se organizan en carpetas de dominio, pruebas, documentos, presentación y evidencias.

Criterio de finalización funcional. Se completaron las cuatro modalidades, los descuentos configurables, los beneficios, las validaciones, la exportación CSV y las pruebas unitarias. El informe describe estos componentes y sus resultados de forma consistente con la implementación.

Eficiencia. Cada liquidación ejecuta una cantidad fija de operaciones del dominio. Con precisión numérica acotada, procesar n empleados requiere tiempo O(n). La exportación de la demostración reúne los n resultados en una lista y utiliza memoria O(n).

Alcance de la versión. La solución implementada funciona por consola y utiliza empleados de ejemplo. La persistencia de datos, el prorrateo de períodos, la interfaz gráfica y la integración con otros sistemas quedan como posibilidades de ampliación.

7. Conclusiones

La programación orientada a objetos permitió representar cuatro formas de remuneración mediante un contrato común. Las clases especializadas conservan sus reglas de cálculo, mientras el servicio coordina la liquidación sin depender de una modalidad concreta.

La separación de responsabilidades y la política configurable facilitan el mantenimiento del sistema. El uso de Decimal y el redondeo por concepto establecen un tratamiento consistente de los importes. Las 25 pruebas aprobadas respaldan los casos de cálculo, las condiciones de frontera y las validaciones evaluadas.

Como continuación del proyecto se propone incorporar captura interactiva y almacenamiento de empleados, reutilizando el dominio y ampliando las pruebas para cubrir esas funciones.

8. Ejecución y repositorio

La aplicación requiere Python 3.10 o superior. El sistema de nómina y sus pruebas funcionan con la biblioteca estándar. Desde la carpeta del proyecto se ejecutan los siguientes comandos:

Operación	**Comando**
Demostración	python -m nomina
Exportación CSV	python -m nomina --csv evidencia/nomina_ejemplo.csv
Pruebas unitarias	python -m unittest discover -s tests -v

Repositorio: https://github.com/tab-Jesus/IngDeSoftware. El código fuente, los registros de pruebas, el reporte de ejemplo y la presentación acompañan el informe en las carpetas del proyecto.

Referencias

[1] *Actividad Unidad Tres.* Guía de trabajo de Ingeniería de Software.

[2] Python Software Foundation. *decimal: Decimal fixed-point and floating-point arithmetic.* https://docs.python.org/3/library/decimal.html

[3] Python Software Foundation. *abc: Abstract Base Classes.* https://docs.python.org/3/library/abc.html

[4] Python Software Foundation. *dataclasses: Data Classes.* https://docs.python.org/3/library/dataclasses.html

[5] Python Software Foundation. *unittest: Unit testing framework.* https://docs.python.org/3/library/unittest.html